Aula — Conceitos

Perceptron de Camada Única

Objetivos desta sessão

- Descrever o perceptron de camada única (modelo, saída em {0; 1} e papel do viés) e situá-lo na história das redes neurais.
- Aplicar à mão a regra de aprendizado do perceptron e acompanhar como pesos e viés mudam a cada erro.
- Interpretar a atualização geometricamente: o vetor de pesos gira em direção ao ponto mal classificado.
- Enunciar o teorema de convergência do perceptron e dizer o que ele garante e o que não garante.
- Explicar por que o perceptron não converge no XOR nem em dados não separáveis, e o que fazer nesses casos.
- Distinguir perceptron e Adaline (regra delta) e situá-los ao lado da regressão linear e da regressão logística.

1. Onde estamos e o que muda hoje

Na aula sobre o neurônio artificial calculamos a saída de um neurônio e escolhemos **à mão** os pesos e o viés que reproduzem as portas AND e OR. Hoje damos o passo que falta: o neurônio **aprende** esses valores sozinho, a partir de exemplos rotulados.

O **perceptron** é exatamente isso: um neurônio com ativação em degrau mais uma regra simples de correção de erro. Ele é o primeiro modelo de rede neural que aprende com dados, e quase tudo o que veremos depois (perceptron de várias camadas, retropropagação) é uma generalização das ideias de hoje.

1.1. Um pouco de história

Ano	Marco	Por que importa
1943	McCulloch e Pitts propõem o neurônio lógico	primeiro modelo matemático de neurônio, com pesos fixos
1958	Rosenblatt publica o perceptron	pela primeira vez os pesos são **aprendidos** dos exemplos
1960	Widrow e Hoff propõem o Adaline; construção do Mark I Perceptron	regra delta (mínimos quadrados) e a primeira implementação em hardware
1962	Novikoff demonstra a convergência do perceptron	garantia de que o treino para, se os dados forem separáveis
1969	Minsky e Papert publicam *Perceptrons*	mostram o limite do modelo (XOR) e frearam a área por anos

> **"Camada única" quer dizer uma camada de pesos ajustáveis**
> O perceptron desta aula liga as entradas diretamente à saída. Não há camadas ocultas. É por isso que a fronteira de decisão é sempre linear, e é esse o limite que vamos explorar na Seção 5.

2. O modelo

O perceptron é o neurônio da aula anterior com a ativação em degrau. Para um exemplo com entradas **x** = (x_1; ...; x_n):

$$z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b = w \cdot x + b$$
$$\hat{y} = 1 \text{ se } z \geq 0; \quad \hat{y} = 0 \text{ caso contrário}$$

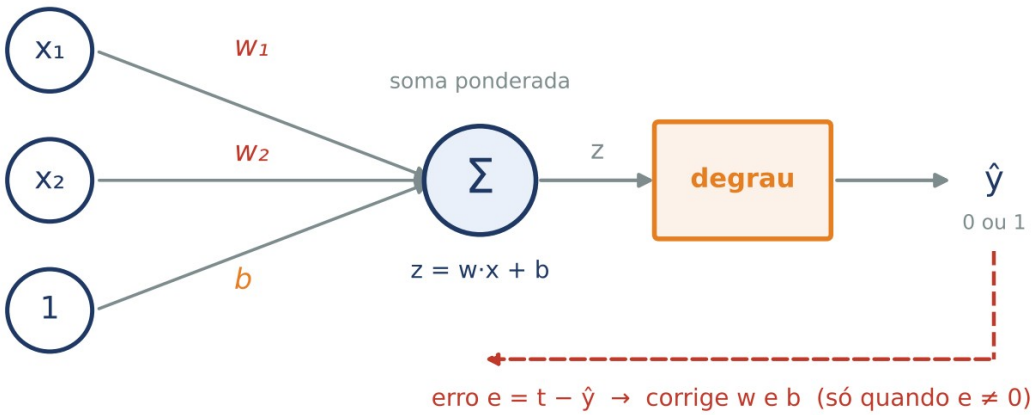


Figura 1 — o perceptron: entradas, soma ponderada com viés, degrau e a realimentação do erro que corrige w e b.

Os parâmetros que o treino ajusta são os pesos **w** e o viés **b**. A **taxa de aprendizado** η (letra grega eta) é um hiperparâmetro: escolhida pelo projetista, controla o tamanho de cada correção. A fronteira de decisão é o conjunto de pontos com z = 0, isto é, a reta (em 2D) ou o hiperplano $w \cdot x + b = 0$.

2.1. O truque do viés

Para escrever a regra de aprendizado em uma linha só, trata-se o viés como um peso comum cuja entrada vale sempre 1: acrescentamos $x_0 = 1$ a cada exemplo e chamamos b de w_0. Assim $z = w \cdot x$, com o viés já embutido. Nesta apostila mantemos w e b separados porque deixa mais claro o papel de cada um, mas as duas formas são equivalentes.

2.2. Rótulos {0; 1} ou {−1; +1}

Com rótulos $t \in \{0; 1\}$, o erro $e = t - \hat{y}$ só pode valer −1, 0 ou +1. Com rótulos $t \in \{-1; +1\}$, o perceptron erra quando $t \cdot z \leq 0$ e a correção é $w \leftarrow w + \eta \cdot t \cdot x$. As duas convenções descrevem o mesmo algoritmo. Usamos {0; 1} na prática e no laboratório, e {−1; +1} no enunciado do teorema de convergência (Seção 4), onde a conta fica mais limpa.

3. A regra de aprendizado

A ideia é a de um professor corrigindo uma prova: o perceptron percorre os exemplos um a um, calcula sua resposta ŷ, compara com o rótulo t e, **só se errou**, corrige os parâmetros:

```
e = t − ŷ
w ← w + η · e · x
b ← b + η · e
```

Situação	e = t − ŷ	O que a regra faz	Efeito em z
acertou (ŷ = t)	0	nada	—
devia ser 1 e deu 0	+1	$w \leftarrow w + \eta\, x$; $b \leftarrow b + \eta$	z **aumenta** em $\eta(\|x\|^2 + 1)$
devia ser 0 e deu 1	−1	$w \leftarrow w - \eta\, x$; $b \leftarrow b - \eta$	z **diminui** em $\eta(\|x\|^2 + 1)$

Repare no efeito: cada correção empurra z **na direção certa**, mas não garante que o mesmo exemplo passe a ser classificado corretamente de uma só vez. O que a convergência garante (Seção 4) é que, repetindo o processo, os erros acabam.

3.1. Intuição geométrica

O vetor **w** é perpendicular à fronteira e aponta para o lado em que o perceptron responde 1. Quando um ponto de classe 1 fica do lado errado, somar x a w faz **w girar em direção a x**, e a fronteira gira junto, passando a deixar o ponto do lado certo. Para um ponto de classe 0 mal classificado, subtrair x afasta w dele.

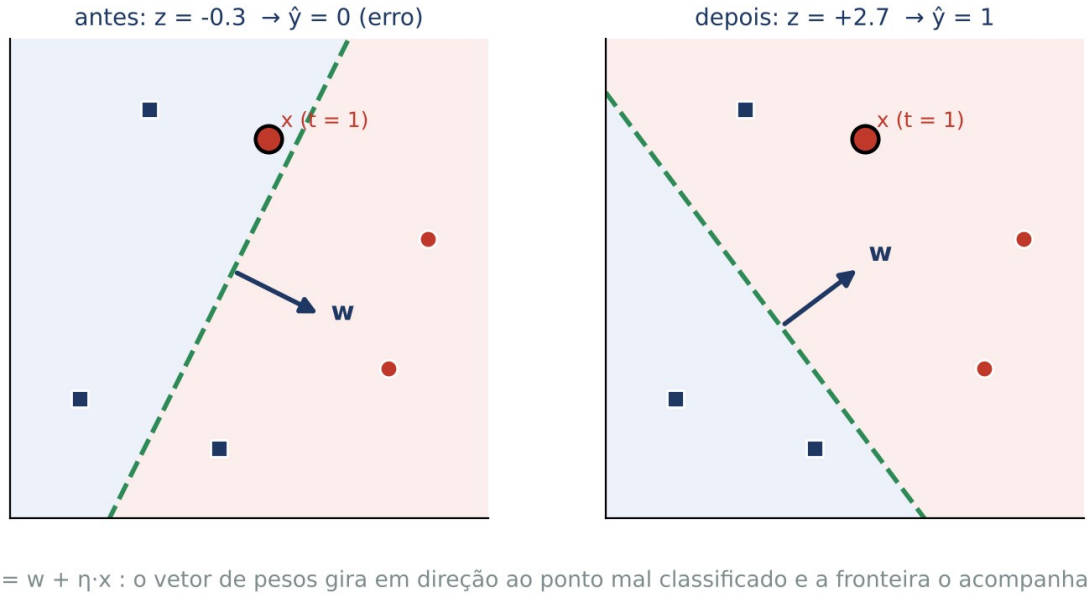


Figura 2 — uma atualização com η = 1: o ponto x (classe 1) estava do lado errado; depois de w' = w + η x a fronteira o acolhe.

3.2. Exemplo feito à mão: a porta AND

Dados: AND, η = 1, w = (0; 0), b = 0, e os exemplos sempre na ordem (0,0), (0,1), (1,0), (1,1). Uma **época** é uma passagem completa pelos quatro exemplos. A tabela mostra a primeira época:

x	t	z = w·x + b	ŷ	e	w depois	b depois
(0; 0)	0	0·0 + 0·0 + 0 = 0	1	−1	(0; 0)	−1
(0; 1)	0	0 + 0 − 1 = −1	0	0	(0; 0)	−1
(1; 0)	0	0 + 0 − 1 = −1	0	0	(0; 0)	−1
(1; 1)	1	0 + 0 − 1 = −1	0	+1	(1; 1)	0

Ao fim da época 1: w = (1; 1), b = 0, com 2 erros. Repetindo o processo, o treino termina na época 6, sem nenhum erro:

Época	Erros	w ao final	b ao final
1	2	(1; 1)	0
2	3	(2; 1)	−1
3	3	(2; 1)	−2
4	2	(2; 2)	−2
5	1	(2; 1)	−3
6	0	(2; 1)	−3

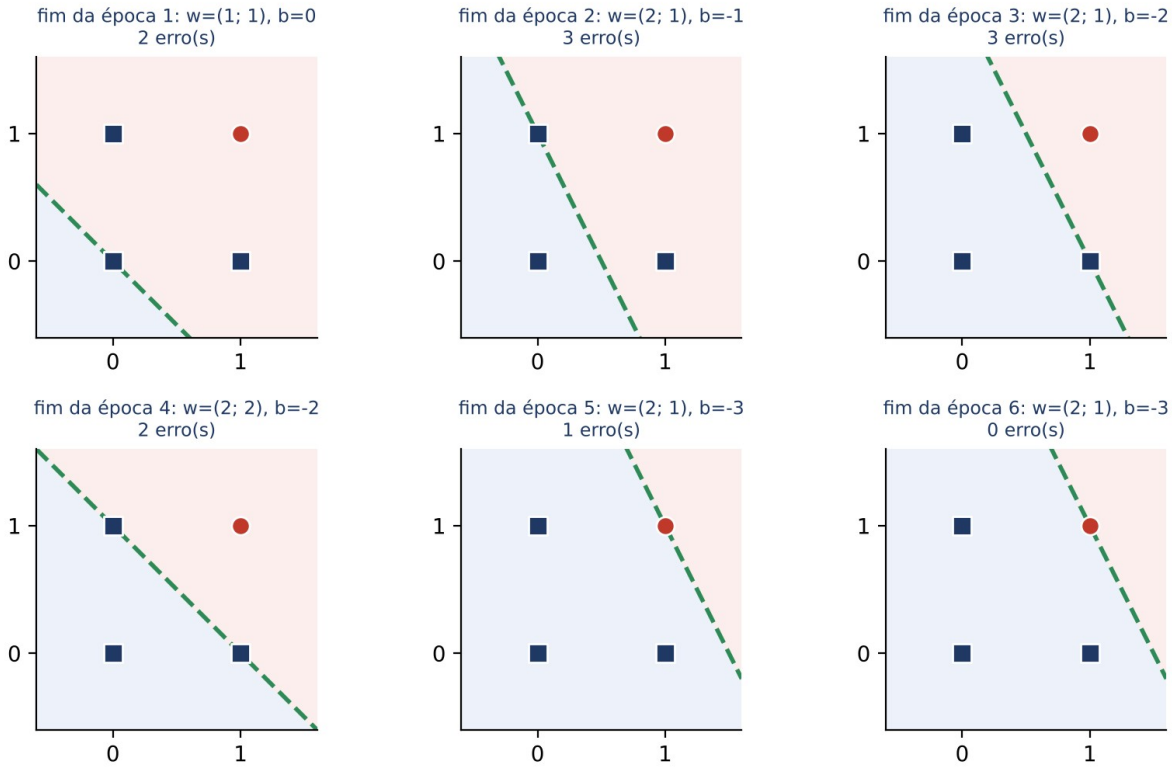


Figura 3 — a fronteira do perceptron ao fim de cada época no AND. Quadrados: saída 0; círculos: saída 1; fundo rosado: região em que o perceptron responde 1.

A fronteira final é $2x_1 + x_2 - 3 = 0$. O ponto (1; 1) cai **exatamente sobre ela** ($z = 0$) e conta como classe 1, porque a regra é $z \geq 0$.

> **Os erros não diminuem em linha reta**
>
> A contagem de erros por época foi 2, 3, 3, 2, 1, 0: subiu antes de cair. O perceptron não minimiza uma função de custo a cada passo; ele apenas corrige o exemplo da vez. Por isso a garantia de convergência (Seção 4) é sobre o treino terminar, não sobre os erros caírem a cada época.

3.3. O algoritmo

```
inicialize w = 0 e b = 0
repita, a cada época:
    erros = 0
    para cada exemplo (x, t), em ordem fixa ou embaralhada:
        z = w · x + b
        ŷ = 1 se z >= 0, senão 0
        e = t - ŷ
        se e != 0:
            w = w + η · e · x
            b = b + η · e
            erros = erros + 1
    se erros == 0: pare          # convergiu
    se época == máximo: pare     # separabilidade não garantida
```

Em NumPy, o treino inteiro cabe em poucas linhas (o laboratório parte exatamente daqui):

```
def treina_perceptron(X, t, eta=1.0, max_epocas=100):
    w, b = np.zeros(X.shape[1]), 0.0
    for epoca in range(max_epocas):
        erros = 0
        for x, alvo in zip(X, t):
            e = alvo - (1 if w @ x + b >= 0 else 0)
            if e != 0:
                w, b, erros = w + eta * e * x, b + eta * e, erros + 1
        if erros == 0:
            break
    return w, b
```

4. Quando o perceptron converge

Um conjunto de dados é **linearmente separável** quando existe alguma reta (ou hiperplano) com todos os pontos de uma classe de um lado e os da outra do lado oposto. A **margem** γ é a distância entre a fronteira separadora e o ponto mais próximo dela, medida com $\|w^*\| = 1$.

> **Teorema de convergência do perceptron (Novikoff, 1962)**
>
> Sejam rótulos $t \in \{-1; +1\}$ e um hiperplano pela origem que separa os dados com margem $\gamma > 0$, isto é, existe w^* com $\|w^*\| = 1$ e $t \cdot (w^* \cdot x) \geq \gamma$ para todo exemplo. Se $\|x\| \leq R$ para todo x, então o perceptron comete **no máximo $(R/\gamma)^2$ erros**, qualquer que seja a ordem dos exemplos.
>
> Com viés, acrescenta-se uma componente constante $x_0 = 1$ a cada x, o que altera R.

Ideia da prova, em três linhas. Seja k o número de erros até agora e w o vetor de pesos (começando em zero). (1) A cada erro, $w \cdot w^*$ cresce pelo menos γ, logo $w \cdot w^* \geq k\gamma$. (2) A cada

erro, ‖w‖² cresce no máximo R² (pois t · (w · x) ≤ 0 num erro), logo ‖w‖ ≤ R√k. (3) Como w · w* ≤ ‖w‖, vem kγ ≤ R√k, ou seja, k ≤ (R/γ)².

> **O que o teorema garante e o que não garante**
>
> **Garante:** se os dados forem linearmente separáveis, o treino termina em um número finito de correções, sem depender de η nem da ordem.
>
> **Não garante:** qual fronteira será encontrada (há infinitas que separam os dados), que ela tenha margem grande, nem que o limite (R/γ)² seja justo: na prática o número de erros costuma ser bem menor. E nada diz sobre dados não separáveis.

O limite depende só da geometria dos dados (R e γ), não do número de exemplos nem da dimensão. O Exercício 4 calcula R, γ e o limite em um caso concreto.

5. Quando o perceptron não converge

5.1. O XOR

Já vimos que nenhuma reta separa as saídas do XOR. O que o perceptron faz ao tentar? A figura compara o número de erros por época em AND, OR e XOR, com os mesmos η e pesos iniciais:

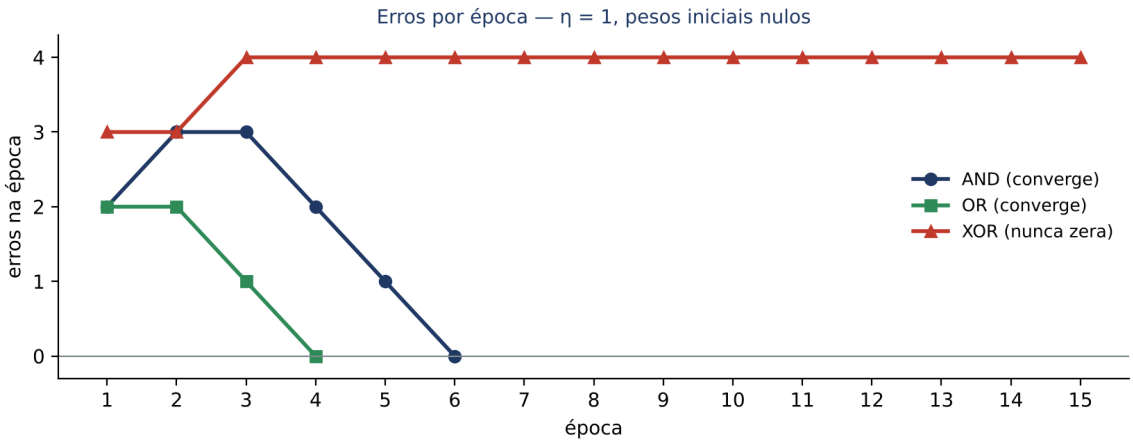


Figura 4 — AND e OR chegam a zero erros; no XOR a contagem fica presa em 3 ou 4 erros por época, para sempre.

No XOR os pesos **não se estabilizam**: eles passam a repetir os mesmos valores em ciclo. Sem um critério de parada extra, o algoritmo rodaria indefinidamente. Por isso todo treino de perceptron precisa de um **número máximo de épocas**.

5.2. Dados reais, ruído e sobreposição

Na prática, as classes quase sempre se sobrepõem um pouco, e dados reais raramente são separáveis. Nesse caso o perceptron continua corrigindo exemplos sem nunca terminar, e **os pesos da última época não são necessariamente os melhores**: um único exemplo ruidoso pode desfazer um bom ajuste em uma só correção. Alternativas:

- **Algoritmo do bolso (pocket):** durante o treino, guardar "no bolso" os pesos que cometeram o menor número de erros no conjunto de treino e devolver esses.
- **Perceptron com média dos pesos:** devolver a média dos pesos ao longo do treino, que é mais estável.

- **Trocar a regra:** o Adaline (Seção 7) minimiza o erro quadrático e se acomoda mesmo sem separabilidade.
- **Aumentar a capacidade do modelo:** camadas ocultas, tema das próximas aulas.

5.3. A ordem dos exemplos

A solução final depende da ordem em que os exemplos são apresentados: ordens diferentes levam a fronteiras diferentes, todas corretas no treino. Na prática, **embaralha-se** os exemplos a cada época, o que também evita ciclos causados por uma ordem desfavorável.

6. Taxa de aprendizado e inicialização

No perceptron, a taxa de aprendizado η se comporta de modo curioso. Com **pesos e viés iniciais nulos**, η apenas multiplica w e b por uma constante: z fica multiplicado por η, o sinal de z não muda, e portanto a sequência de erros e as decisões do perceptron são **as mesmas** para qualquer $\eta > 0$. O Exercício 5 mostra isso no OR com $\eta = 1$ e $\eta = 0{,}5$.

Com pesos iniciais **não nulos**, η passa a importar, porque a escala das correções é comparada à escala dos pesos iniciais: o percurso de treino e a solução final mudam.

> **Um detalhe de computador**
>
> A invariância acima vale em aritmética exata. Em ponto flutuante, valores como 0,1 não são representados exatamente, e os empates $z = 0$, que são comuns em exemplos de portas lógicas, podem ser decididos de forma diferente por causa do arredondamento. Com $\eta = 1$ ou $\eta = 0{,}5$ (potências de 2) a conta é exata. Veremos isso no laboratório.

Já no Adaline, que usa o valor de z e não só o sinal, η **importa muito**, como mostra a seção seguinte.

7. Do perceptron ao Adaline

Em 1960, Widrow e Hoff propuseram o **Adaline** (ADAptive LInear NEuron). A diferença está em **qual saída entra no erro**: no perceptron, o erro usa a saída já passada pelo degrau ($\hat{y}$); no Adaline, usa a soma ponderada z, ainda real. A decisão final continua sendo um degrau sobre z; como o alvo é 0 ou 1, o limiar fica no ponto médio, $z \geq 0{,}5$.

```
perceptron:  e = t − ŷ    (ŷ ∈ {0; 1})
Adaline:     e = t − z    (z real; regra delta)
em ambos:    w ← w + η · e · x
```

Como $e = t - z$ é um número real, o Adaline corrige **em todo exemplo**, proporcionalmente ao tamanho do erro, e não só quando erra. Essa regra é a descida do gradiente para o erro quadrático médio, a mesma ideia da regressão linear. Por isso o Adaline converge **mesmo sem separabilidade**, desde que η seja pequeno o bastante.

	Perceptron	Adaline	Regressão logística
Saída usada no erro	degrau: $\hat{y} \in \{0; 1\}$	identidade: z	sigmoide: $\sigma(z)$
Corrige quando	só ao errar	em todo exemplo	em todo exemplo
O que otimiza	nenhuma função de custo (regra de correção)	erro quadrático médio	entropia cruzada
Dados não separáveis	não converge	converge ao melhor ajuste linear	converge ao melhor ajuste
Papel de η	irrelevante com w inicial nulo	decisivo (estabilidade)	decisivo

Cuidado com η no Adaline

Numa atualização isolada, o erro de z encolhe pelo fator $(1 - \eta(\|x\|^2 + 1))$. Se $\eta(\|x\|^2 + 1) > 1$, z **ultrapassa** o alvo; se passa de 2, o erro cresce e o treino diverge. O Exercício 8 mostra uma ultrapassagem que inverte a classe de um exemplo.

Por isso se **padronizam os atributos** (StandardScaler) antes de treinar um Adaline: $\|x\|$ grande exige η minúsculo.

8. Mais de duas classes

Um perceptron separa duas classes. Para K classes usa-se a estratégia **um contra todos**: um perceptron por classe, cada um aprendendo "é a classe k ou não é", e a classe prevista é a do neurônio com maior z. A fronteira entre quaisquer duas classes continua linear. No laboratório trabalharemos com pares de classes.

9. Treino versus generalização

Convergir significa **zero erros no conjunto de treino**, e nada mais. Um perceptron que separa o treino perfeitamente pode errar em dados novos, sobretudo se os dados têm ruído ou a margem é pequena. Como nas aulas de validação, a avaliação honesta é feita em dados **separados** do treino (holdout ou validação cruzada), com acurácia e matriz de confusão. Quanto maior a margem, melhor tende a ser a generalização, uma ideia que reaparece nas máquinas de vetores de suporte.

No scikit-learn, `sklearn.linear_model.Perceptron` implementa o modelo desta aula, já com embaralhamento dos exemplos a cada época e critério de parada.

10. No laboratório

No laboratório que continua esta aula você vai implementar o perceptron em NumPy e acompanhar o treino época a época; ver a fronteira se mover até separar os dados; verificar numericamente o limite de erros do teorema; provocar o fracasso no XOR e em dados sobrepostos e usar o algoritmo do bolso; comparar o perceptron com o Adaline; e avaliar tudo na Iris, com separação treino e teste. O perceptron deve ser seu, escrito em NumPy; o scikit-learn entra só na conferência final.

11. Exercícios resolvidos

Cinco exercícios resolvidos passo a passo, cada um com a ideia-chave ao final. Estude-os antes de resolver os da Seção 12: eles usam os mesmos raciocínios, com outros números.

Exercício 1 — Treinar um perceptron à mão (OR)

Treine um perceptron para a porta OR com η = 1, w = (0; 0), b = 0 e os exemplos sempre na ordem (0,0), (0,1), (1,0), (1,1). **(a)** Monte a tabela da primeira época. **(b)** Continue até uma época sem erros e informe quantas épocas foram necessárias e quantos erros ocorreram no total. **(c)** Confira a solução nas quatro linhas da tabela-verdade e escreva a equação da fronteira.

Resolução.

(a) Em cada exemplo: calcular z, aplicar o degrau, comparar com t e corrigir só se errou.

x	t	z	ŷ	e	w depois	b depois
(0; 0)	0	0	1	−1	(0; 0)	−1
(0; 1)	1	0 + 0 − 1 = −1	0	+1	(0; 1)	0
(1; 0)	1	0 + 0 + 0 = 0	1	0	(0; 1)	0
(1; 1)	1	0 + 1 + 0 = 1	1	0	(0; 1)	0

(b) Repetindo o processo, época a época:

Época	Erros	w ao final	b ao final
1	2	(0; 1)	0
2	2	(1; 1)	0
3	1	(1; 1)	−1
4	0	(1; 1)	−1

São 4 épocas (a última só confirma que não há mais erros) e 2 + 2 + 1 = 5 erros no total.

(c) Com $z = x_1 + x_2 − 1$: em (0; 0), z = −1 e ŷ = 0; em (0; 1) e (1; 0), z = 0 e ŷ = 1; em (1; 1), z = 1 e ŷ = 1. As quatro linhas do OR estão corretas. A fronteira é a reta $x_1 + x_2 = 1$, e os pontos (0; 1) e (1; 0) caem exatamente sobre ela.

Ideia-chave. O perceptron só mexe nos pesos quando erra, e a solução encontrada é uma entre infinitas possíveis.

Exercício 2 — Ler um perceptron treinado

Um perceptron treinado ficou com w = (2; −1) e b = −1. **(a)** Classifique A(1; 0), B(0; 1), C(2; 2) e D(0,5; 0). **(b)** Escreva a equação da fronteira e diga onde ela cruza os eixos. **(c)** Qual dos pontos, A ou B, está mais longe da fronteira, e a que distância? (A distância de x à fronteira é |z| / ‖w‖.)

Resolução.

(a) Calcule $z = 2x_1 − x_2 − 1$ em cada ponto:

Ponto	z	ŷ
A (1; 0)	2·1 − 0 − 1 = 1	1
B (0; 1)	0 − 1 − 1 = −2	0
C (2; 2)	4 − 2 − 1 = 1	1
D (0,5; 0)	1 − 0 − 1 = 0	1 (sobre a fronteira)

(b) A fronteira é $2x_1 - x_2 - 1 = 0$, ou $x_2 = 2x_1 - 1$. Com $x_2 = 0$ temos $x_1 = 0{,}5$; com $x_1 = 0$ temos $x_2 = -1$. Ela cruza os eixos em (0,5; 0) e (0; −1). O ponto D é justamente o cruzamento com o eixo x_1, por isso z = 0.

(c) $\|w\| = \sqrt{(4 + 1)} = \sqrt{5} \approx 2{,}236$. Distância de A: $|1| / 2{,}236 \approx 0{,}447$. Distância de B: $|-2| / 2{,}236 \approx 0{,}894$. O ponto B está mais longe, a cerca de 0,894.

Ideia-chave. O sinal de z decide a classe; o valor $|z| / \|w\|$ mede a distância geométrica à fronteira. Pontos com z = 0 caem em $\hat{y} = 1$ pela regra $z \geq 0$.

Exercício 3 — Uma atualização: perceptron versus Adaline

Um neurônio tem w = (0,5; −0,5) e b = 0,2. Chega o exemplo x = (1; 2) com t = 1. Use $\eta = 0{,}1$. **(a)** Calcule z e ŷ. **(b)** Aplique a regra do perceptron e recalcule z. **(c)** Aplique a regra delta do Adaline (erro t − z) e recalcule z. **(d)** Compare os dois resultados.

Resolução.

(a) $z = 0{,}5 \cdot 1 + (-0{,}5) \cdot 2 + 0{,}2 = 0{,}5 - 1{,}0 + 0{,}2 = -0{,}3$. Como $z < 0$, $\hat{y} = 0$. O rótulo é 1, então $e = t - \hat{y} = +1$: houve erro.

(b) Perceptron: $w \leftarrow w + \eta \cdot e \cdot x = (0{,}5 + 0{,}1 \cdot 1 \cdot 1;\ -0{,}5 + 0{,}1 \cdot 1 \cdot 2) = (0{,}6;\ -0{,}3)$; $b \leftarrow 0{,}2 + 0{,}1 = 0{,}3$. Novo $z = 0{,}6 \cdot 1 + (-0{,}3) \cdot 2 + 0{,}3 = 0{,}6 - 0{,}6 + 0{,}3 = 0{,}3$, e agora $\hat{y} = 1$.

(c) Adaline: o erro é $t - z = 1 - (-0{,}3) = 1{,}3$. Então $w \leftarrow w + \eta \cdot 1{,}3 \cdot x = (0{,}5 + 0{,}13;\ -0{,}5 + 0{,}26) = (0{,}63;\ -0{,}24)$ e $b \leftarrow 0{,}2 + 0{,}13 = 0{,}33$. Novo $z = 0{,}63 \cdot 1 + (-0{,}24) \cdot 2 + 0{,}33 = 0{,}63 - 0{,}48 + 0{,}33 = 0{,}48$.

(d) Nos dois casos z subiu, mas por quantidades diferentes: +0,6 no perceptron ($= 0{,}1 \cdot 1 \cdot 6$, com $\|x\|^2 + 1 = 6$) e +0,78 no Adaline ($= 0{,}1 \cdot 1{,}3 \cdot 6$). O perceptron corrigiu a classe com o passo mínimo e, uma vez acertado o exemplo, não mexe mais nos pesos. O Adaline não olha a classe, olha a distância de z ao alvo: com z = 0,48 ainda faltam 0,52 para o alvo 1 (e, com o limiar 0,5, o exemplo continua na classe 0), então ele segue puxando z nos próximos passos, e seguiria puxando mesmo que z já estivesse do lado certo.

Ideia-chave. O perceptron corrige só quando erra e por um passo de tamanho fixo; o Adaline corrige sempre, proporcionalmente ao erro de z.

Exercício 4 — Limite de erros do perceptron

Quatro pontos com rótulos em {−1; +1}: P1 (3; 1) com +1, P2 (1; 3) com +1, P3 (−1; −1) com −1 e P4 (−2; 0) com −1. O vetor $w^* = (1; 1)/\sqrt{2}$ os separa por um hiperplano pela origem. **(a)** Calcule a margem $\gamma = \min t \cdot (w^* \cdot x)$ e o raio $R = \max \|x\|$. **(b)** Qual é o limite $(R/\gamma)^2$ de erros? **(c)** Rode o perceptron com $\eta = 1$, w = (0; 0) e ordem P1, P2, P3, P4 (erro quando $t \cdot (w \cdot x) \leq 0$; correção $w \leftarrow w + t \cdot x$) e conte os erros reais.

Resolução.

(a) Margens $t \cdot (w^* \cdot x)$: P1: $(3 + 1)/\sqrt{2} \approx 2{,}828$; P2: $(1 + 3)/\sqrt{2} \approx 2{,}828$; P3: $-(-1 - 1)/\sqrt{2} \approx 1{,}414$; P4: $-(-2)/\sqrt{2} \approx 1{,}414$. A menor é **$\gamma = \sqrt{2} \approx 1{,}414$**. As normas são $\|P1\| = \|P2\| = \sqrt{10}$, $\|P3\| = \sqrt{2}$ e $\|P4\| = 2$, logo **$R = \sqrt{10} \approx 3{,}162$**.

(b) $(R/\gamma)^2 = 10 / 2 =$ **5** erros, no máximo.

(c) Com w = (0; 0): P1 dá t·(w·x) = 0 ≤ 0, erro, e w = (0; 0) + 1·(3; 1) = (3; 1). P2: 1·(3·1 + 1·3) = 6 > 0, ok. P3: −(−3 − 1) = 4 > 0, ok. P4: −(−6) = 6 > 0, ok. Na segunda passada não há erros. O perceptron cometeu **1 erro**, bem abaixo do limite de 5.

Ideia-chave. O limite $(R/\gamma)^2$ vale para qualquer ordem e depende só da geometria, não do número de exemplos. É uma garantia de pior caso, quase sempre pessimista.

Exercício 5 — Taxa de aprendizado e inicialização

Refaça o treino do OR do Exercício 1 (mesma ordem dos exemplos) com **(a)** η = 0,5 e pesos iniciais nulos; **(b)** η = 1, w = (0,3; −0,2) e b = 0,1. **(c)** Compare ambos com o Exercício 1.

Resolução.

(a) Com w e b nulos no início, cada correção vale agora metade da anterior, e z fica pela metade, com o mesmo sinal. A sequência de erros é idêntica à do Exercício 1:

Época	Erros	w ao final	b ao final
1	2	(0; 0,5)	0
2	2	(0,5; 0,5)	0
3	1	(0,5; 0,5)	−0,5
4	0	(0,5; 0,5)	−0,5

(b) Com pesos iniciais não nulos o caminho muda. Na primeira época: (0; 0): z = 0,1 → ŷ = 1, erro (e = −1), b = −0,9; (0; 1): z = −1,1 → ŷ = 0, erro (e = +1), w = (0,3; 0,8) e b = 0,1; (1; 0) e (1; 1) estão corretos. Continuando:

Época	Erros	w ao final	b ao final
1	2	(0,3; 0,8)	0,1
2	2	(0,3; 1,8)	0,1
3	2	(1,3; 1,8)	0,1
4	1	(1,3; 1,8)	−0,9
5	0	(1,3; 1,8)	−0,9

(c) Em (a) os pesos finais são metade dos do Exercício 1, (0,5; 0,5) e b = −0,5, a fronteira $x_1 + x_2 = 1$ é a mesma e os 5 erros também. Em (b) os pesos finais são outros, (1,3; 1,8) e b = −0,9, a fronteira é diferente e são 7 erros. Todas as três soluções implementam o OR.

Ideia-chave. Com pesos iniciais nulos, η só muda a escala de w e b, não as decisões. Com pesos iniciais não nulos, o caminho e a solução mudam, mas o problema continua resolvido, pois o OR é separável.

12. Exercícios

Cada exercício vale 2,0 pontos. Mostre as contas: a resposta sem o raciocínio vale a metade.

Exercício 6 — Treinar um perceptron à mão (NOR) (2,0)

Treine um perceptron para a porta NOR (saída 1 apenas quando as duas entradas valem 0) com η = 1, w = (0; 0), b = 0 e os exemplos na ordem (0,0), (0,1), (1,0), (1,1). **(a)** Monte a tabela da primeira época. **(b)** Continue até uma época sem erros: quantas épocas foram necessárias e quantos erros

ocorreram no total? **(c)** Escreva a equação da fronteira final e confira as quatro linhas da tabela-verdade.

Exercício 7 — Ler um perceptron treinado (2,0)

Um perceptron treinado ficou com w = (1; −2) e b = 3. **(a)** Classifique P(2; 1), Q(−1; 2) e R(0; 1,5). **(b)** Escreva a equação da fronteira e diga onde ela cruza os eixos; o que você nota sobre o ponto R? **(c)** Qual dos pontos P ou Q está mais longe da fronteira, e a que distância?

Exercício 8 — Perceptron versus Adaline: o perigo de η grande (2,0)

Um neurônio tem w = (−0,4; 0,6) e b = 0,1. Chega o exemplo x = (2; −1) com t = 0. Use η = 0,3. **(a)** Calcule z e ŷ: o perceptron atualiza os pesos? **(b)** Aplique a regra delta do Adaline e recalcule z. Com o limiar z ≥ 0,5, o que aconteceu com a classe prevista? **(c)** Explique o que ocorreu e diga qual η faria o novo z coincidir exatamente com o alvo. (Dica: o erro de z é multiplicado por $1 - \eta(\|x\|^2 + 1)$.)

Exercício 9 — Limite de erros do perceptron (2,0)

Quatro pontos com rótulos em {−1; +1}: P1 (1; 2) com +1, P2 (−1; 3) com +1, P3 (2; −1) com −1 e P4 (−3; −2) com −1. O vetor w* = (0; 1) os separa por um hiperplano pela origem. **(a)** Calcule a margem γ e o raio R. **(b)** Qual é o limite $(R/\gamma)^2$? **(c)** Rode o perceptron com η = 1, w = (0; 0) e ordem P1, P2, P3, P4 (erro quando $t \cdot (w \cdot x) \leq 0$; correção $w \leftarrow w + t \cdot x$) e conte os erros reais. Compare com o limite.

Exercício 10 — Taxa de aprendizado e inicialização (2,0)

Refaça o treino do NOR do Exercício 6 (mesma ordem dos exemplos) com **(a)** η = 0,5 e pesos iniciais nulos; **(b)** η = 1, w = (−0,2; 0,3) e b = −0,1. **(c)** Compare ambos com o Exercício 6: o que muda e o que não muda? Todos os treinos resolvem o NOR?

13. Leitura recomendada

- ROSENBLATT, Frank. The perceptron: a probabilistic model for information storage and organization in the brain. Psychological Review, v. 65, n. 6, 1958.
- NOVIKOFF, Albert B. J. On convergence proofs on perceptrons. Symposium on the Mathematical Theory of Automata, 1962.
- WIDROW, Bernard; HOFF, Marcian E. Adaptive switching circuits. IRE WESCON Convention Record, 1960.
- MINSKY, Marvin; PAPERT, Seymour. Perceptrons: an introduction to computational geometry. MIT Press, 1969.
- BRAGA, Antonio de Padua; CARVALHO, Andre Ponce de Leon F. de; LUDERMIR, Teresa Bernarda. Redes neurais artificiais: teoria e aplicações. LTC, 2007.
- SILVA, Ivan Nunes da; SPATTI, Danilo Hernane; FLAUZINO, Rogerio Andrade. Redes neurais artificiais para engenharia e ciências aplicadas. Artliber, 2010.
- GÉRON, Aurélien. Mãos à obra: aprendizado de máquina com Scikit-Learn, Keras e TensorFlow. Alta Books.